B. Always Changing

time limit per test 1.5 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1100 | Tags: dp, greedy, math, strings | https://codeforces.com/problemset/problem/2252/B

You are given a binary string s of length n.

A string is called alternating if no two adjacent characters are the same. For example, `0101`, `1`, and `01` are alternating, but `0110` is not.

You want to transform s into an alternating string by performing the following operation any number of times (possibly zero):

- Choose any character currently in the string and delete it.

However, your sequence of operations must follow a rule: the characters you delete must strictly alternate. This means if the last character you deleted was `0`, the next character you delete must be `1`, and vice versa. Your very first deleted character can be either `0` or `1`.

Find the minimum number of operations required to make s an alternating string. If it is impossible to achieve this, output -1.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \le n \le 2 \cdot 10^5$) — the length of the string s.

The second line of each test case contains the binary string s of length n, consisting only of the characters `0` and `1`.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer — the minimum number of operations required to make s an alternating string, or -1 if it is impossible.

Example

Input
5 4 0101 3 111 6 100110 6 100010 6 011110

Output

```
0
-1
2
3
5
```

Note

In the first test case, the string `0101` is already alternating. No operations are needed, so the answer is 0.

In the second test case, the string is `111`. To make it alternating, we must leave at most one `1`, which requires deleting two `1`s. However, our deletions must strictly alternate between `0` and `1`. Since we have no `0`s to delete, it is impossible to perform two deletions. The answer is -1.

In the third test case, the string is `100110`. We can simply delete one `0` and one `1` to resolve the adjacent duplicates. Since we deleted exactly one `0` and one `1`, the operations are perfectly balanced and valid. The remaining string is `1010`, requiring exactly 2 operations.

In the fourth test case, the string is `100010`. To resolve the adjacent duplicates, we would naturally want to delete two `0`s to form `1010`. However, deleting two `0`s and zero `1`s violates the strict alternation rule (the difference in counts cannot exceed 1). To fix this imbalance, we are forced to additionally delete a `1` from the edge of the string. We delete two `0`s and one `1` (e.g., in the order `0`, `1`, `0`), leaving the alternating string `010`. This requires 3 operations.

In the fifth test case, the string is `011110`. Resolving the adjacent duplicates requires deleting three `1`s. To balance this, we must additionally delete at least two `0`s. Since our compressed string `010` only has two `0`s (at its outer edges), we are forced to delete both of them, leaving a final string of just `1`. We deleted three `1`s and two `0`s, which takes 5 operations.